

- ✓ Google Colab Notebook Code
- ✓ 1. Setup Environment and Verify GPU

Installs Ultralytics (YOLOv8), ONNX exporter, and dependencies.

```
!pip install ultralytics onnx onnxruntime opencv-python-headless matplotlib pillow

import torch
import os
from google.colab import files

print("--- ENVIRONMENT CHECK ---")
print(f"PyTorch Version: {torch.__version__}")
if torch.cuda.is_available():
    print(f"GPU Active: {torch.cuda.get_device_name(0)}")
else:
    print("WARNING: GPU not detected. Go to Runtime > Change runtime type > T4 GPU.")
```

```

Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cuspars-cu12==12.5.8.93.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: httpx<1,>=0.28 in /usr/local/lib/python3.13/dist-packages (from ultralytics-platform)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.13/dist-packages (from cuda-bindings)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->ultralytics-platform)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.13->ultralytics-platform)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->torch)
Downloading ultralytics-8.4.128-py3-none-any.whl (1.4 MB)
----- 1.4/1.4 MB 76.4 MB/s eta 0:00:00
Downloading onnx-1.22.0-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (19.1 MB)
----- 19.1/19.1 MB 100.3 MB/s eta 0:00:00
Downloading onnxruntime-1.29.0-cp313-cp313-manylinux_2_28_x86_64.whl (23.1 MB)
----- 23.1/23.1 MB 86.5 MB/s eta 0:00:00
Downloading ultralytics_platform-0.1.14-py3-none-any.whl (65 kB)
----- 65.3/65.3 kB 6.7 MB/s eta 0:00:00
Downloading ultralytics_thop-2.1.6-py3-none-any.whl (30 kB)
Installing collected packages: onnxruntime, onnx, ultralytics-platform, ultralytics-thop, ultralytics
Successfully installed onnx-1.22.0 onnxruntime-1.29.0 ultralytics-8.4.128 ultralytics-platform-0.1.14 ultralytics-thop-2.1.6
--- ENVIRONMENT CHECK ---
PyTorch Version: 2.11.0+cu128
GPU Active: Tesla T4

```

✓ 2. Create Dataset Directory Structure

This cell sets up the directory layout and creates dummy images so the entire notebook is runnable out-of-the-box.

```
import os
import numpy as np
from PIL import Image

# Define root dataset directory
DATASET_DIR = "/content/selfie_helmet_dataset"
CLASSES = ["full_face_helmet", "half_face_helmet", "no_helmet"]

# Create Train and Validation folders
for split in ["train", "val"]:
    for cls in CLASSES:
        os.makedirs(os.path.join(DATASET_DIR, split, cls), exist_ok=True)

# Generate dummy images so you can test training right now
def generate_dummy_images(folder, count=10):
    for cls in CLASSES:
        target_path = os.path.join(folder, cls)
        for i in range(count):
            # Create a simple colored random image (224x224)
            img_data = np.random.randint(0, 255, (224, 224, 3), dtype=np.uint8)
            img = Image.fromarray(img_data)
            img.save(os.path.join(target_path, f"sample_{i}.jpg"))

generate_dummy_images(os.path.join(DATASET_DIR, "train"), count=20)
generate_dummy_images(os.path.join(DATASET_DIR, "val"), count=5)

print(f"Dataset structure ready at: {DATASET_DIR}")
print("Folders created for: full_face_helmet, half_face_helmet, no_helmet")

Dataset structure ready at: /content/selfie_helmet_dataset
Folders created for: full_face_helmet, half_face_helmet, no_helmet
```

3. Train the YOLO Classification Model

Fine-tunes the lightweight YOLOv8 nano classifier (`yolo8n-cls`) on your selfie classes.

```
from ultralytics import YOLO

# 1. Load pre-trained weights
model = YOLO('yolo8n-cls.pt')

# 2. Train on your selfie dataset
# Change epochs to 30-50 when training on your actual real images
results = model.train(
    data=DATASET_DIR,
    epochs=5,           # Increase for real dataset
    imgsz=224,          # Standard input resolution for classification
    batch=16,
    project='helmet_project',
    name='selfie_classifier',
    exist_ok=True
)

print("\nTraining completed successfully!")
```

```

Epoch   GPU_mem   loss   Instances   Size
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100%
1/5      0.25G     1.128      12      224: 100% 4/4 2.3it/s 1.7s
      classes top1_acc top5_acc: 100% 1/1 7.9it/s 0.1s
      all      0.333      1

Epoch   GPU_mem   loss   Instances   Size
2/5      0.254G     1.083      12      224: 100% 4/4 32.0it/s 0.1s
      classes top1_acc top5_acc: 100% 1/1 9.2it/s 0.1s
      all      0.333      1

Epoch   GPU_mem   loss   Instances   Size
3/5      0.254G     1.127      12      224: 100% 4/4 31.1it/s 0.1s
      classes top1_acc top5_acc: 100% 1/1 11.3it/s 0.1s
      all      0.333      1

Epoch   GPU_mem   loss   Instances   Size
4/5      0.254G     1.099      12      224: 100% 4/4 32.8it/s 0.1s
      classes top1_acc top5_acc: 100% 1/1 10.1it/s 0.1s
      all      0.333      1

Epoch   GPU_mem   loss   Instances   Size
5/5      0.254G     1.076      12      224: 100% 4/4 30.9it/s 0.1s
      classes top1_acc top5_acc: 100% 1/1 11.8it/s 0.1s
      all      0.4      1

```

5 epochs completed in 0.001 hours.

Optimizer stripped from /content/runs/classify/helmet_project/selfie_classifier/weights/last.pt, 3.0MB

Optimizer stripped from /content/runs/classify/helmet_project/selfie_classifier/weights/best.pt, 3.0MB

Validating /content/runs/classify/helmet_project/selfie_classifier/weights/best.pt...

Ultralytics 8.4.128 🚀 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

YOLOv8n-cls summary (fused): 30 layers, 1,438,723 parameters, 0 gradients, 3.3 GFLOPs

train: /content/selfie_helmet_dataset/train... found 60 images in 3 classes ✓

val: /content/selfie_helmet_dataset/val... found 15 images in 3 classes ✓

test: None...

```

      classes top1_acc top5_acc: 100% 1/1 17.4it/s 0.1s
      all      0.4      1

```

Speed: 0.1ms preprocess, 0.3ms inference, 0.0ms loss, 0.0ms postprocess per image

Results saved to /content/runs/classify/helmet_project/selfie_classifier

Training completed successfully!

✓ 4. Verify Inference Logic (True / False Output)

Tests the trained model on an image, applies confidence thresholds, and formats the output into a production JSON response.

```
import os
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

# Correct path generated by Ultralytics classification trainer
WEIGHTS_PATH = "/content/runs/classify/helmet_project/selfie_classifier/weights/best.pt"

# Fallback to last.pt if best.pt isn't found
if not os.path.exists(WEIGHTS_PATH):
    WEIGHTS_PATH = "/content/runs/classify/helmet_project/selfie_classifier/weights/last.pt"

print(f"Loading model weights from: {WEIGHTS_PATH}")
trained_model = YOLO(WEIGHTS_PATH)

def verify_selfie(image_path, confidence_threshold=0.65):
    # Run classification
    results = trained_model(image_path, verbose=False)

    # Get top prediction class ID and confidence score
    top1_index = results[0].probs.top1
    predicted_class = results[0].names[top1_index]
    confidence = float(results[0].probs.top1conf)

    # Business logic: True if full or half helmet with sufficient confidence
    helmet_worn = (predicted_class in ["full_face_helmet", "half_face_helmet"]) and (confidence >= co

    return {
        "helmet_worn": helmet_worn,
        "detected_class": predicted_class,
        "confidence": round(confidence, 4)
```

```
}  
  
# Test on a validation sample  
test_image_path = "/content/selfie_helmet_dataset/val/full_face_helmet/sample_0.jpg"  
  
response = verify_selfie(test_image_path)  
  
print("\n--- INFERENCE RESULT ---")  
print(response)  
  
# Display image  
img = cv2.imread(test_image_path)  
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
plt.title(f"Helmet Worn: {response['helmet_worn']} ({response['detected_class']} - {response['confide  
plt.imshow(img)  
plt.axis("off")  
plt.show()
```

```
Loading model weights from: /content/runs/classify/helmet_project/selfie_classifier/weights/best.pt
```

```
--- INFERENCE RESULT ---
```

```
{'helmet_worn': False, 'detected_class': 'full_face_helmet', 'confidence': 0.4753}
```

Helmet Worn: False (full_face_helmet - 47.5%)



✓ 5. Export Model for Server Deployment

Exports the trained PyTorch model (`best.pt`) to ONNX format (`best.onnx`) and triggers a file download to your computer.

```
import os
from ultralytics import YOLO
from google.colab import files
```

```
WEIGHTS_PATH = "/content/runs/classify/helmet_project/selfie_classifier/weights/best.pt"
```

```
if not os.path.exists(WEIGHTS_PATH):  
    WEIGHTS_PATH = "/content/runs/classify/helmet_project/selfie_classifier/weights/last.pt"  
  
export_model = YOLO(WEIGHTS_PATH)  
  
# 1. Export to ONNX format for lightweight server inference  
onnx_file_path = export_model.export(format="onnx")  
print(f"ONNX Model saved to: {onnx_file_path}")  
  
# 2. Trigger browser downloads for both files  
print("Downloading weight files to your computer...")  
files.download(WEIGHTS_PATH)  
files.download(onnx_file_path)
```

Ultralytics 8.4.128 🚀 Python-3.13.15 torch-2.11.0+cu128 CPU (Intel Xeon CPU @ 2.00GHz)
 💡 ProTip: Export to OpenVINO format for best performance on Intel hardware. Learn more at <https://docs.ultralytics.com>
 YOLOv8n-cls summary (fused): 30 layers, 1,438,723 parameters, 0 gradients, 0.4 GFLOPs

PyTorch: starting from '/content/runs/classify/helmet_project/selfie_classifier/weights/best.pt' with input sha
requirements: Ultralytics requirement ['onnxslim>=0.1.82'] not found, attempting AutoUpdate...
 Using Python 3.13.15 environment at: /usr
 Resolved 10 packages in 243ms
 Prepared 2 packages in 41ms
 Installed 2 packages in 6ms
 + colorama==0.4.6
 + onnxslim==0.1.96

requirements: AutoUpdate success ✅ 0.6s
 WARNING ⚠️ **requirements:** Restart runtime or rerun command for updates to take effect

ONNX: starting export with onnx 1.22.0 opset 18...
ONNX: slimming with onnxslim 0.1.96...
ONNX: export success ✅ 1.3s, saved as '/content/runs/classify/helmet_project/selfie_classifier/weights/best.onnx'

Export complete (1.4s)
 Results saved to /content/runs/classify/helmet_project/selfie_classifier/weights/best.onnx
 Predict: yolo predict task=classify model=/content/runs/classify/helmet_project/selfie_classifier/weights/best.onnx
 Validate: yolo val task=classify model=/content/runs/classify/helmet_project/selfie_classifier/weights/best.onnx
 Visualize: <https://netron.app>
 ONNX Model saved to: /content/runs/classify/helmet_project/selfie_classifier/weights/best.onnx
 Downloading weight files to your computer...

✓ Production Backend Server Setup (No Retraining Needed)

Once you download `best.pt` or `best.onnx` from Colab, you **never need Colab or training code again**. Place the weight file into your server project directory alongside this deployment script.

Backend Implementation (`server.py`)

Here is a ready-to-use production server script powered by **FastAPI** that loads your exported model once into memory and handles incoming photo requests instantly:

```
from fastapi import FastAPI, UploadFile, File, HTTPException
from ultralytics import YOLO
import cv2
import numpy as np

app = FastAPI(title="Selfie Helmet Verification API")

# Load model weights into RAM once when server starts
# Replace "best.pt" with your actual exported file path
MODEL_PATH = "best.pt"
CONFIDENCE_THRESHOLD = 0.70

try:
    model = YOLO(MODEL_PATH)
    print("Selfie Helmet Classifier loaded successfully into server memory.")
except Exception as e:
    raise RuntimeError(f"Failed to load model file: {e}")

@app.post("/verify-helmet")
async def verify_helmet(file: UploadFile = File(...)):
    # Validate file format
    if not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="File uploaded must be an image.")

    # 1. Convert incoming file stream to OpenCV image array
    contents = await file.read()
    np_array = np.frombuffer(contents, np.uint8)
    img = cv2.imdecode(np_array, cv2.IMREAD_COLOR)

    if img is None:
        raise HTTPException(status_code=400, detail="Invalid or corrupt image data.")

    # 2. Perform Inference
    results = model(img, verbose=False)
```

```
top1_index = results[0].probs.top1
predicted_class = results[0].names[top1_index]
confidence = float(results[0].probs.top1conf)

# 3. Decision Logic
is_helmet_valid = (
    predicted_class in ["full_face_helmet", "half_face_helmet"]
) and (confidence >= CONFIDENCE_THRESHOLD)

# 4. Return API Response
return {
    "status": "success",
    "helmet_worn": is_helmet_valid,
    "details": {
        "predicted_class": predicted_class,
        "confidence": round(confidence, 4),
        "threshold_applied": CONFIDENCE_THRESHOLD
    }
}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

How to Run the Server

1. Install dependencies on your server: